# Assignment-8

Name:U.Sushmitha

Batch:24

Ht.No:2303A51705

Task Description #1 (Username Validator – Apply AI in Authentication Context)

• Task: Use AI to generate at least 3 assert test cases for a function is_valid_username(username) and then implement the function using Test-Driven Development principles.

• Requirements:

o Username length must be between 5 and 15 characters.

o Must contain only alphabets and digits.

o Must not start with a digit.

o No spaces allowed. Example Assert Test Cases:

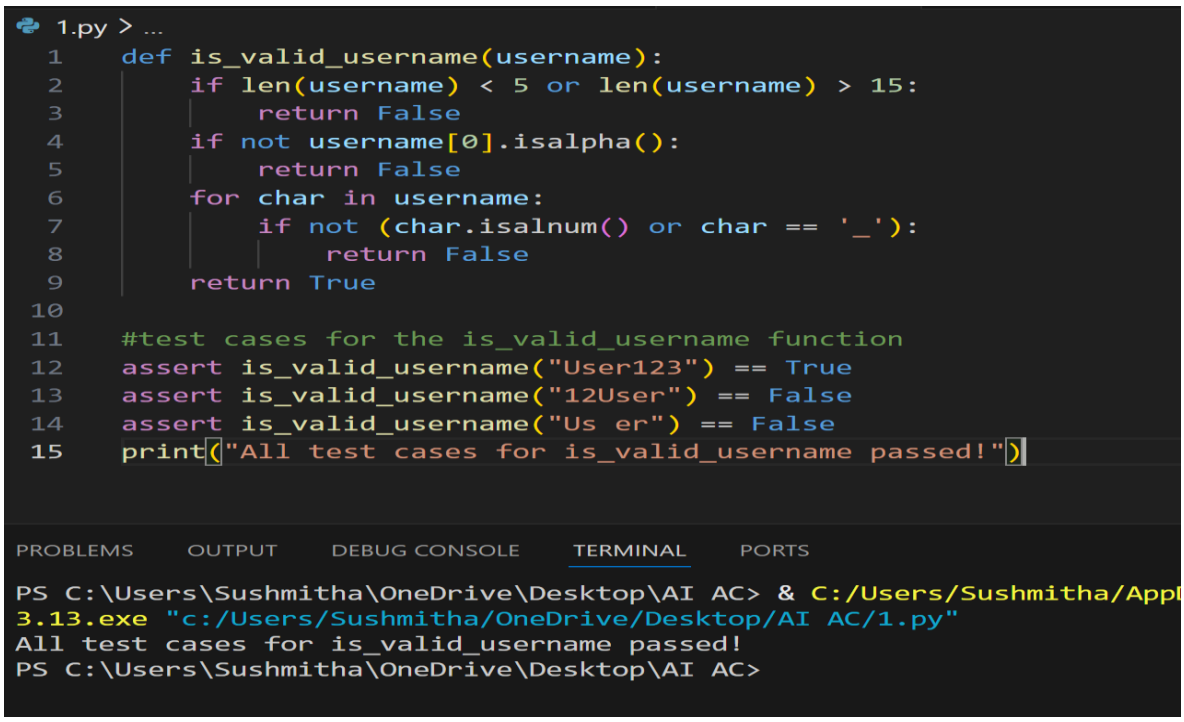
assert is_valid_username("User123") == True

assert is_valid_username("12User") == False assert

is_valid_username("Us er") == False

Expected Output #1:

• Username validation logic successfully passing all AI-generated test cases.



Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

- Task: Use AI to generate at least 3 assert test cases for a function classify_value(x) and implement it using conditional logic and loops.
- Requirements:

o If input is an integer, classify as "Even" or "Odd".

o If input is 0, return "Zero".

o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

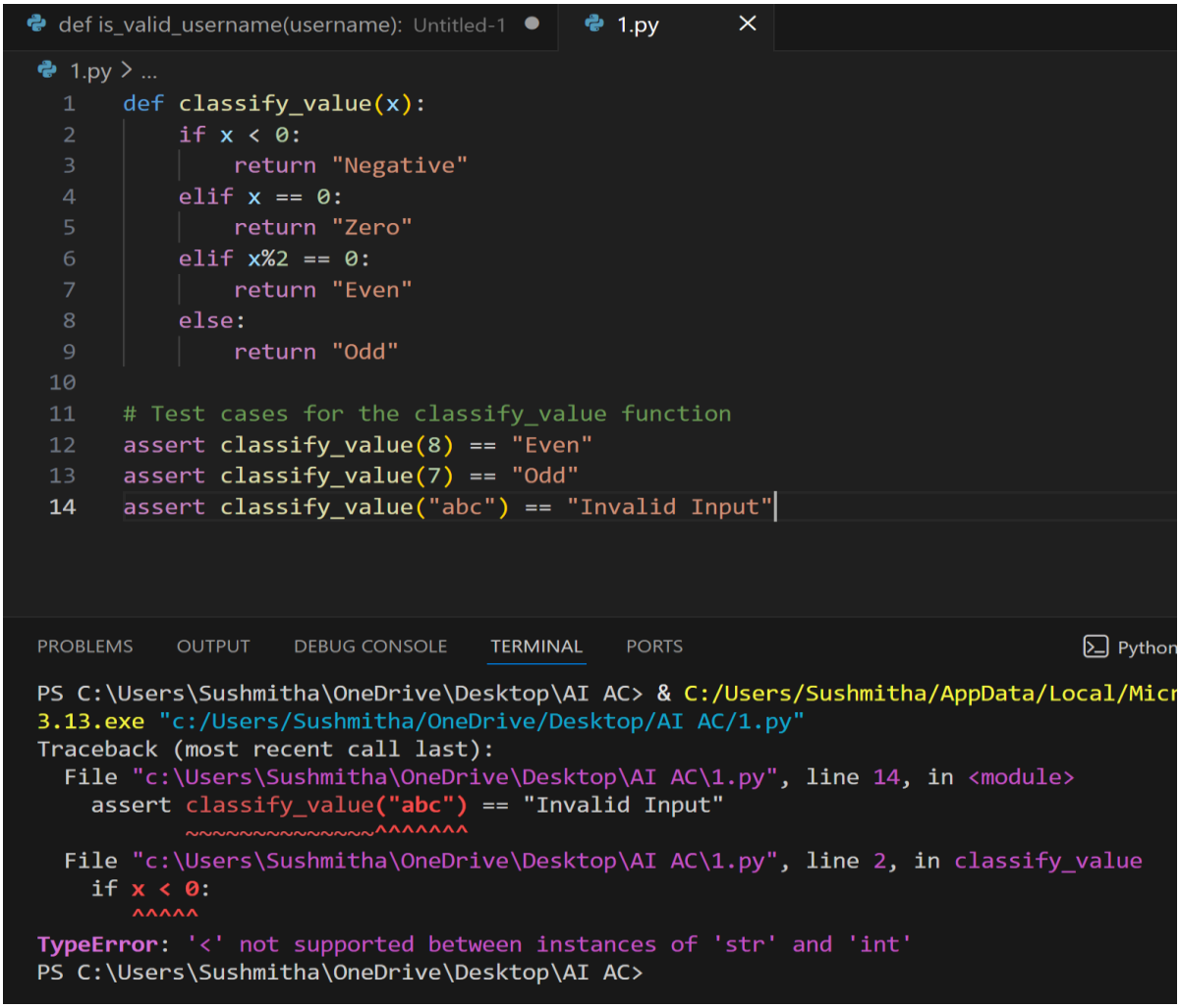
assert classify_value(8) == "Even" assert

classify_value(7) == "Odd" assert

classify_value("abc") == "Invalid Input"

Expected Output #2:

- Function correctly classifying values and passing all test cases.



Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

- Task: Use AI to generate at least 3 assert test cases for a function is_palindrome(text) and implement the function.
- Requirements:

o Ignore case, spaces, and punctuation. o Handle edge cases

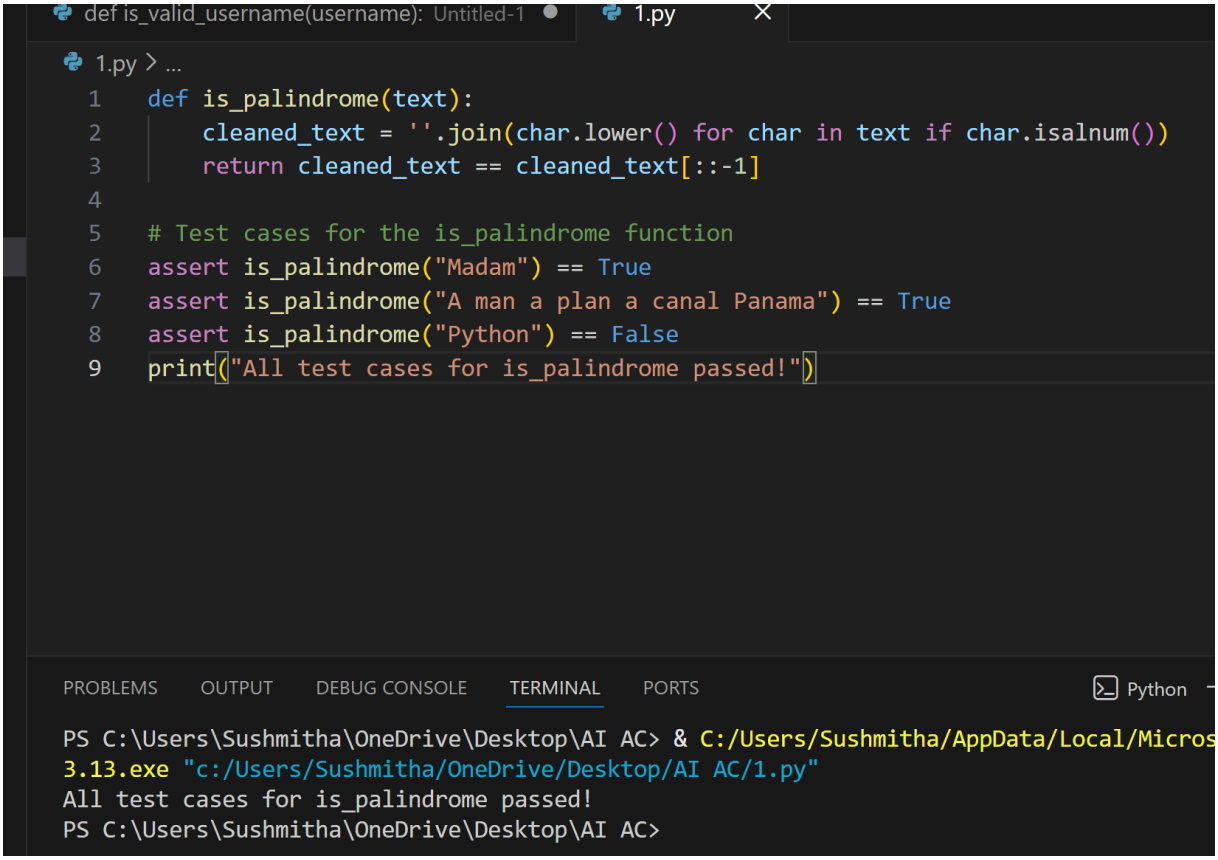
such as empty strings and single characters.

Example Assert Test Cases:

assert is_palindrome("Madam") == True assert

is_palindrome("A man a plan a canal Panama") ==True

assert is_palindrome("Python") == False




Task Description #4 (Email ID Validation – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for a function validate_email(email) and implement the function.
- Requirements:

o Must contain @ and . o Must not start or end

with special characters. o Should handle invalid

formats gracefully.

Example Assert Test Cases:

assert validate_email("user@example.com") == True

assert validate_email("userexample.com") == False assert

validate_email("@gmail.com") == False

Expected Output #5:

- Email validation function passing all AI-generated test cases and handling edge cases correctly.

```python
def validate_email(email):
    if '@' not in email or '.' not in email:
        return False
    at_index = email.index('@')
    dot_index = email.rindex('.')
    if at_index < 1 or dot_index < at_index + 2 or dot_index >= len(email) - 1:
        return False
    return True

# Test cases for the validate_email function
assert validate_email("user@example.com") == True
assert validate_email("userexample.com") == False
assert validate_email("@gmail.com") == False
print("All test cases for validate_email passed!")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  Python

PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/Microsoft/Wir
3.13.exe "c:/Users/Sushmitha/OneDrive/Desktop/AI AC/1.py"
All test cases for validate_email passed!
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

Task 5 (Perfect Number Checker – Test Case Design)

- Function: Check if a number is a perfect number (sum of divisors = number).
- Test Cases to Design:

o Normal case: 6 → True,

10 → False. o Edge case: 1.

o Negative number case. o

Larger case: 28.

• Requirement: Validate correctness with assertions.

calculater.py > is_perfect

```python
#generate a program to check whether a given number is perfect or not
def is_perfect(n):
    if n < 1:
        return False
    divisors_sum=sum(i for i in range(1, n) if n % i == 0)
    return divisors_sum == n
#test cases for the is_perfect function
assert is_perfect(6) == True
assert is_perfect(28) == True
assert is_perfect(12) == False
assert is_perfect(1) == False
print("All test cases passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/Mi
sktop/AI AC/calculater.py"
All test cases passed!
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

Task 6 (Abundant Number Checker – Test Case Design)

• Function: Check if a number is abundant (sum of divisors >number).

• Test Cases to Design:

o Normal case: 12 → True, 15 → False. o Edge case: 1. o Negative number case.

o Large case: 945.

Requirement: Validate correctness with unittest

```python
def Abundant_Number_Checker(num):
    if num < 1:
        return False
    divisors_sum = sum(i for i in range(1, num) if num % i == 0)
    return divisors_sum > num
import unittest
class TestAbundantNumberChecker(unittest.TestCase):
    def test_abundant(self):
        self.assertTrue(Abundant_Number_Checker(12))
        self.assertTrue(Abundant_Number_Checker(15))
        self.assertTrue(Abundant_Number_Checker(1))
        self.assertFalse(Abundant_Number_Checker(-1))
        self.assertTrue(Abundant_Number_Checker(945))
if __name__ == '__main__':
    unittest.main()
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python3
op/AI ASSISTANT CODING/lab-08(continuation).py"
F
======================================================================
FAIL: test_abundant (__main__.TestAbundantNumberChecker.test_abundant)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "c:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING\lab-08(continuation).py", line 59, in test_abundant
    self.assertTrue(Abundant_Number_Checker(15))
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: False is not true

----------------------------------------------------------------------
Ran 1 test in 0.001s

FAILED (failures=1)
```

Task 7 (Deficient Number Checker – Test Case Design)

• Function: Check if a number is deficient (sum of divisors <number).

• Test Cases to Design:

o Normal case: 8 → True,

12 → False. o Edge case: 1.

o Negative number case. o

Large case: 546.

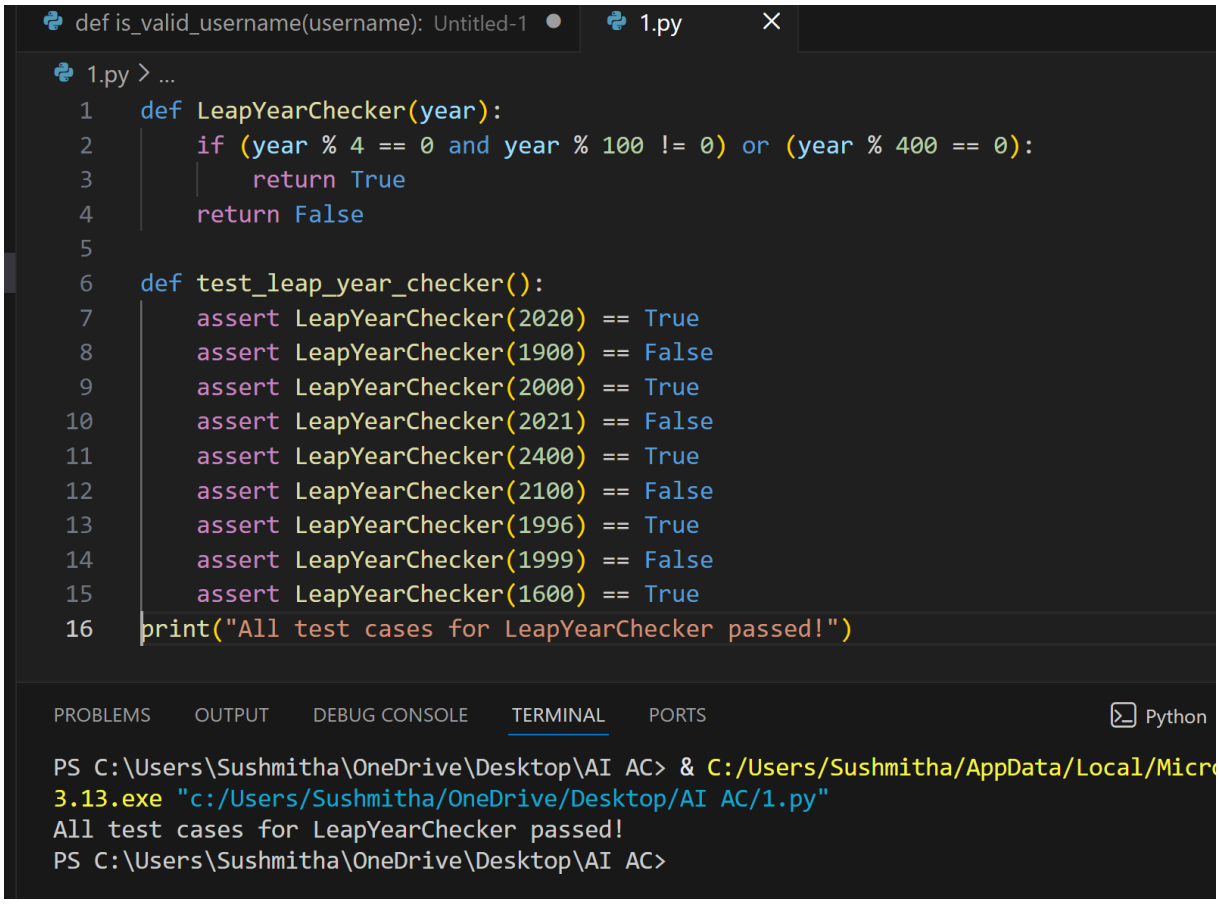
Requirement: Validate correctness with pytest

```python
def deficient_number_checker(num):
    if num < 1:
        return False
    divisors_sum = sum(i for i in range(1, num) if num % i == 0)
    return divisors_sum < num

def test_deficient_number_checker():
    assert deficient_number_checker(8) == True
    assert deficient_number_checker(12) == False
    assert deficient_number_checker(1) == True
    assert deficient_number_checker(546) == False
print("All test cases for deficient_number_checker passed!")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/
3.13.exe "c:/Users/Sushmitha/OneDrive/Desktop/AI AC/1.py"
All test cases for deficient_number_checker passed!
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

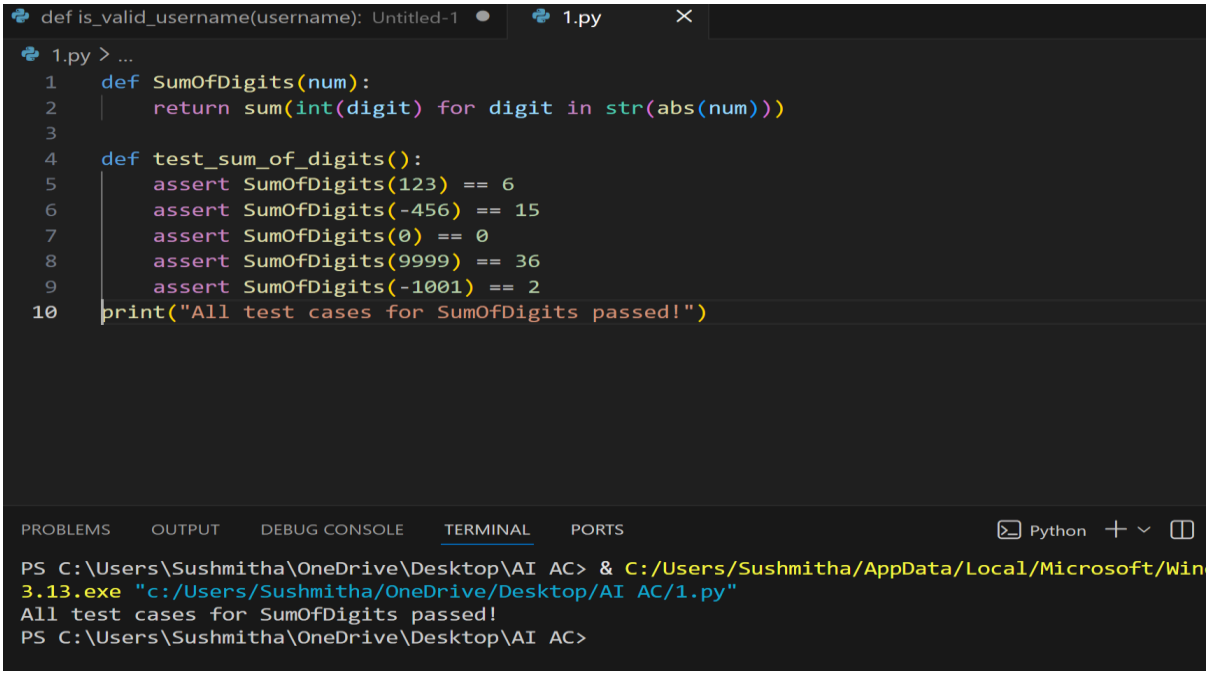
Task 8 :

Write a function LeapYearChecker and validate its implementation using 10 pytest test cases



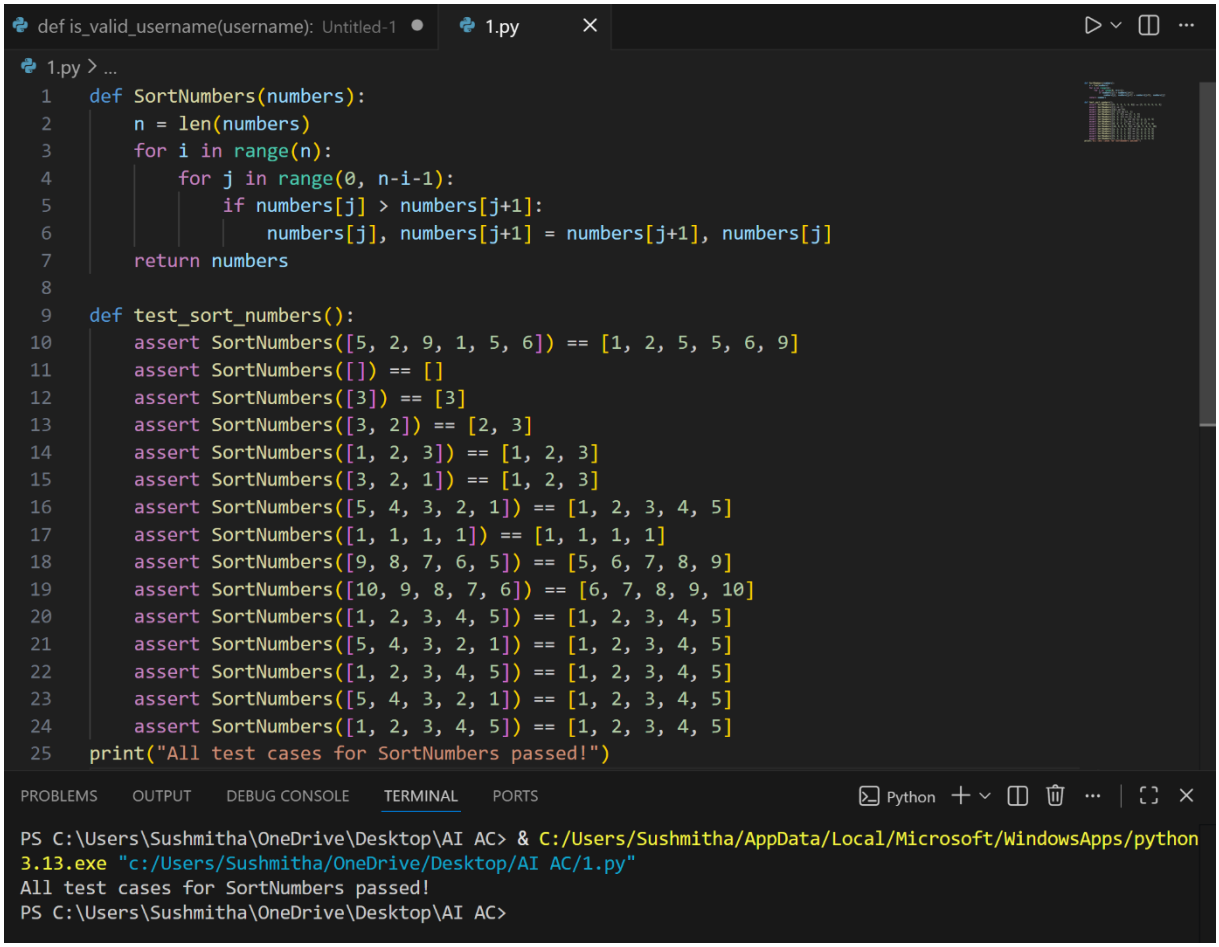

Task 9 :

Write a function SumOfDigits and validate its implementation using 7 pytest test cases.



Task 10 :

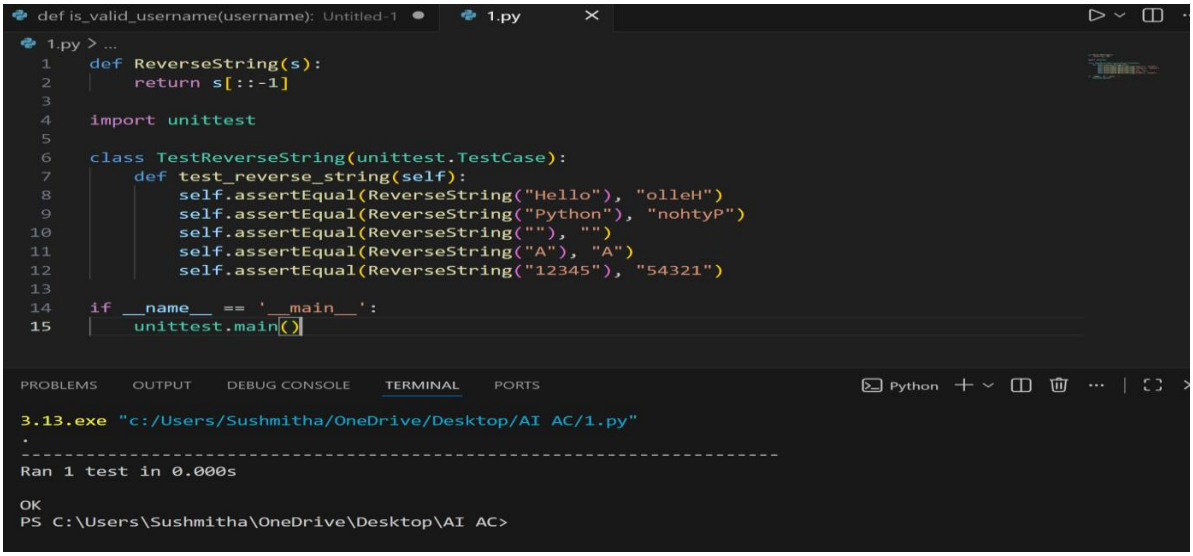
Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.



```
def is_valid_username(username): Untitled-1 ● 1.py ×
1.py > ...
1  def SortNumbers(numbers):
2      n = len(numbers)
3      for i in range(n):
4          for j in range(0, n-i-1):
5              if numbers[j] > numbers[j+1]:
6                  numbers[j], numbers[j+1] = numbers[j+1], numbers[j]
7      return numbers
8
9  def test_sort_numbers():
10     assert SortNumbers([5, 2, 9, 1, 5, 6]) == [1, 2, 5, 5, 6, 9]
11     assert SortNumbers([]) == []
12     assert SortNumbers([3]) == [3]
13     assert SortNumbers([3, 2]) == [2, 3]
14     assert SortNumbers([1, 2, 3]) == [1, 2, 3]
15     assert SortNumbers([3, 2, 1]) == [1, 2, 3]
16     assert SortNumbers([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
17     assert SortNumbers([1, 1, 1, 1]) == [1, 1, 1, 1]
18     assert SortNumbers([9, 8, 7, 6, 5]) == [5, 6, 7, 8, 9]
19     assert SortNumbers([10, 9, 8, 7, 6]) == [6, 7, 8, 9, 10]
20     assert SortNumbers([1, 2, 3, 4, 5]) == [1, 2, 3, 4, 5]
21     assert SortNumbers([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
22     assert SortNumbers([1, 2, 3, 4, 5]) == [1, 2, 3, 4, 5]
23     assert SortNumbers([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
24     assert SortNumbers([1, 2, 3, 4, 5]) == [1, 2, 3, 4, 5]
25 print("All test cases for SortNumbers passed!")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS        Python

PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python
3.13.exe "c:/Users/Sushmitha/OneDrive/Desktop/AI AC/1.py"
All test cases for SortNumbers passed!
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

Task 11 :

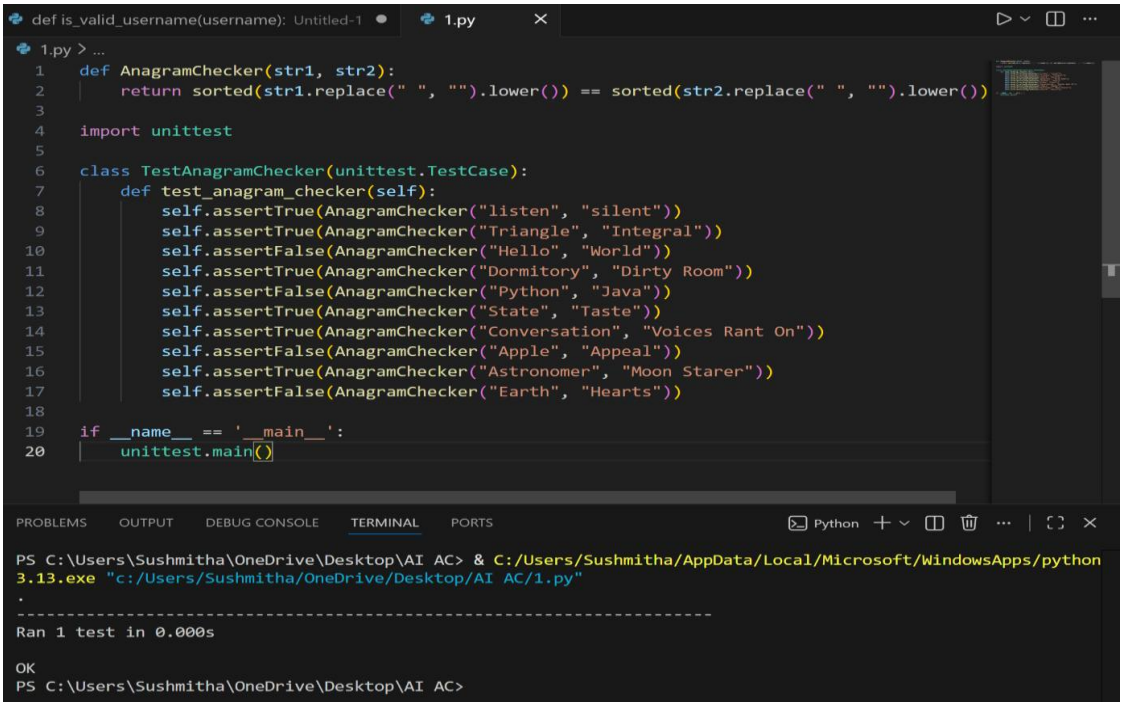
Write a function ReverseString and validate its implementation using 5 unittest test cases



```
def is_valid_username(username): Untitled-1 ● 1.py ×
1.py > ...
1  def ReverseString(s):
2      return s[::-1]
3
4  import unittest
5
6  class TestReverseString(unittest.TestCase):
7      def test_reverse_string(self):
8          self.assertEqual(ReverseString("Hello"), "olleH")
9          self.assertEqual(ReverseString("Python"), "nohtyP")
10         self.assertEqual(ReverseString(""), "")
11         self.assertEqual(ReverseString("A"), "A")
12         self.assertEqual(ReverseString("12345"), "54321")
13
14 if __name__ == '__main__':
15     unittest.main()

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS        Python

3.13.exe "c:/Users/Sushmitha/OneDrive/Desktop/AI AC/1.py"
.
----------------------------------------------------------------------
Ran 1 test in 0.000s

OK
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

Task 12 :

Write a function AnagramChecker and validate its implementation using 10 unittest test cases.




Task 13 :

Write a function ArmstrongChecker and validate its implementation using 8 unittest test cases.